

CineSeat

Real-Time Movie Theatre
Seat Booking System

**Development Roadmap and
Implementation Strategy**

1. Introduction and Objective

This document defines the development roadmap and implementation strategy for CineSeat, a real-time movie theatre seat booking system supporting multiple theatres and multiple concurrent shows. It builds directly on the Week 1 Software Requirements Document (functional and non-functional requirements, and the concurrency and locking strategy defined in Section 3.4) and the Week 2 System Architecture Document (the ten-module breakdown, the layered monolith-first design, and the technology stack rationale). Where the earlier documents describe what the system must do and how it is structured, this document describes the order in which the system will be built, the milestones that mark progress, the methodology used to manage the work, and the risks that could affect delivery.

The roadmap is scoped to the internship's fixed six-week duration rather than to an open-ended timeline. Weeks 1 and 2 have already been spent on Planning and Design; that leaves four working weeks (Weeks 3–6) to take the system from a bootstrapped project through to a tested, deployed build. Each of those four weeks is mapped to one implementation sprint, and each sprint is deliberately aligned with that same week's internship documentation deliverable, so the coding work and the written deliverable for a given week reinforce each other rather than running as two disconnected tracks.

Because four weeks is a materially tighter window than a typical production timeline, this plan treats CineSeat as a scoped MVP: the core booking and concurrency-control flows are treated as non-negotiable, while peripheral hardening work is explicitly deferred and logged as stretch goals rather than silently dropped. That distinction is made explicit in Section 3.1.

2. Development Stage Planning

The project is divided into five logical stages. Planning and Design correspond to the work already completed in Weeks 1 and 2 of the internship; Coding, Testing, and Deployment form the implementation roadmap covered by this document.

Stage	Primary Focus	Key Outputs	Status
Planning	Requirements gathering, scope definition, DSA-to-feature mapping	Software Requirements Document (Week 1)	Complete
Design	System architecture, module boundaries, concurrency model, tech stack	System Architecture Document (Week 2)	Complete
Coding	Iterative implementation of backend modules, seat engine, booking flow, payments	Working Spring Boot services, incremental feature builds	Upcoming
Testing	Unit, integration, concurrency, and load testing against SRD requirements	Test plans, automated test suites, defect log	Upcoming
Deployment	Environment setup, containerization, release, monitoring	Deployed application, deployment runbook	Upcoming

3. Milestone Schedule

The full six-week internship calendar is the schedule of record. Weeks 1–2 (Planning and Design) are already complete. Weeks 3–6 are organized into four one-week sprints — Sprint 1 through Sprint 4 — each ending with a working, demonstrable increment, and each intentionally aligned with that week's internship documentation deliverable.

3.1 Scope Adjustment for the Six-Week Window

A four-week implementation window is too short to build and harden every capability in the Week 1 SRD to production depth. To keep the plan honest and feasible, scope is split into a fixed MVP core, which Sprints 1–4 are committed to delivering, and a set of stretch items that are explicitly deferred beyond the internship rather than quietly dropped.

MVP Core (in scope, Weeks 3–6)	Deferred / Stretch (post-internship)
Seat grid, availability lookup, sliding-window scan Redisson distributed locking and TTL-based hold expiry Reservation state machine and VIP/senior-citizen waitlist Payment integration against a sandbox gateway Core unit, integration, and concurrency test suites Containerized deployment of a single environment	Horizontal scaling / multi-instance load testing Full monitoring and alerting stack Admin dashboard and reporting features Extended JMeter load testing at production-scale volume CI/CD pipeline beyond a basic build-and-test workflow

3.2 Sprint-Level Milestones

Sprint	Week	Milestone	Deliverable / Exit Criteria
Sprint 1	Week 3	Foundation and seat engine	Spring Boot skeleton, PostgreSQL/Redis provisioned, base entity models migrated; 2D seat grid and HashMap-backed status lookup; catalog and availability APIs; sliding-window scan for consecutive seats. This week's document: Development Roadmap (this document).
Sprint 2	Week 4	Concurrency, hold, and waitlist	ReentrantLock in-process locking and Redisson distributed lock integration; TTL-based hold expiry with scheduled cleanup; reservation state machine (HELD → CONFIRMED → EXPIRED); PriorityQueue-based VIP/senior-citizen waitlist with lazy heap deletion. This week's document: Testing Strategy — built directly around validating this sprint's concurrency logic.
Sprint 3	Week 5	Payment, notifications, and integration testing	Sandbox payment gateway integration; idempotent payment confirmation; hold-to-booking conversion; seat-unavailable notification flow; integration and concurrency test suites executed against Sprints 1–2 output. This week's document: Code Review

			— covering the seat engine, locking layer, and booking flow built so far.
Sprint 4	Week 6	Hardening, deployment, and close-out	Bug-fix pass against defects found in Sprint 3 testing; regression suite re-run; containerized deployment to a single environment; deployment runbook; final demo build. This week's document: Retrospective — reviewing what shipped against the MVP scope fixed in Section 3.1.

3.3 Major Milestones Summary

Milestone	Target	Significance
M1 — Seat engine functional	End of Sprint 1 (Week 3)	Seat grid, availability, and multi-show/multi-theatre browsing usable end-to-end
M2 — Concurrency-safe holds	End of Sprint 2 (Week 4)	Core differentiator of the project; validates the locking strategy defined in SRD Section 3.4, tested the same week
M3 — Booking flow complete	End of Sprint 3 (Week 5)	Full user journey — browse, hold, pay, confirm — is functional and integration-tested
M4 — MVP release	End of Sprint 4 (Week 6)	System hardened, deployed, and demo-ready for the internship close-out and for a professional portfolio

4. Implementation Strategy

CineSeat will be developed using an Agile, Scrum-influenced methodology adapted for a single developer, rather than a Waterfall approach. Waterfall was considered and rejected as the primary strategy for this project.

4.1 Why Agile Over Waterfall

The core risk in a seat-booking system is not requirements ambiguity — the requirements were fixed in Week 1 — it is technical risk in the concurrency and locking layer, which is difficult to fully validate on paper. An Agile approach allows the highest-risk components (distributed locking, hold expiry, seat contention) to be built and tested early, in Sprint 2, rather than left until the end of a short linear sequence, so that design flaws surface while there is still time to revise the approach within the six-week window. Waterfall's stronger points — predictable long-range scheduling and heavy upfront documentation — are less valuable here because the requirements and architecture are already fixed outputs of Weeks 1 and 2, and because a rigid sequential plan leaves no room to absorb a slipped estimate inside a four-week build. What remains is primarily an execution and validation problem under a hard deadline, which iterative delivery with weekly checkpoints is better suited to.

4.2 Sprint Structure

Each one-week sprint follows a fixed internal cadence, compressed from a team Scrum process to fit both a single-developer workflow and the internship's weekly documentation cycle:

- **Sprint Planning (Monday):** Define the sprint goal, break it into tasks, and set exit criteria against the SRD and architecture document.
- **Daily Build Log (each working day):** A short written log of what was completed, what is blocked, and what is next — replacing the stand-up meeting for a solo project, and doubling as a defensible record of engineering decisions. With only five working days per sprint, this log is also the earliest warning signal for slippage.
- **Mid-Sprint Check-in (Wednesday):** Review progress against the sprint goal; if behind, trim scope against the deferred list in Section 3.1 rather than letting the sprint run over.
- **Sprint Review (Friday):** Demo the working increment against the exit criteria defined at sprint planning, and against that week's internship deliverable.
- **Sprint Retrospective (Friday):** Record what worked, what didn't, and one concrete process change to carry into the next sprint.

4.3 Branching and Integration Workflow

Development follows a trunk-based Git workflow with short-lived feature branches, one per task, merged into main only after the module's own unit tests pass locally and the CI pipeline is green. This keeps main always in a demonstrable state, which matters for a portfolio project that may be reviewed at any point.

5. Resource Estimation

As a solo-developer project constrained to a six-week internship, resourcing is expressed in developer-days per stage rather than team headcount. Weeks 3–6 provide four five-day working weeks — twenty developer-days total — for Coding, Testing, and Deployment combined; the estimates below are sized to fit inside that fixed budget rather than derived independently and then reconciled against it.

Stage	Effort (dev-days)	Primary Challenge	Mitigation
Planning	10 (complete)	Scoping DSA concepts to realistic features without over-engineering	Mapped each DSA concept to one concrete feature (Section 4 of SRD) rather than adding speculative ones
Design	10 (complete)	Choosing monolith vs. microservices without over-committing early	Adopted monolith-first design with explicit future migration path, documented and defensible
Coding	12 (Sprints 1–3)	Correctly implementing distributed locking and TTL expiry inside a single sprint, not several	Locking and hold-expiry work scheduled in Sprint 2, the second of only four sprints, so problems surface with sprints still in reserve to fix them
Testing	5 (Sprints 3–4)	Fitting concurrency validation into days, not weeks, as a single developer	Correctness-focused concurrency tests prioritized over broad-scale load tests, which are pushed to the deferred list in Section 3.1
Deployment	3 (Sprint 4)	Limited prior DevOps/containerization experience under a one-week deadline	Single-environment Docker Compose setup only — no orchestration platform — kept

			deliberately minimal so it fits inside Sprint 4
--	--	--	---

Because the twenty-day budget is tight, testing effort is still weighted above what a typical CRUD application would need — seat-hold concurrency is the project's core technical claim, and even a compressed schedule keeps dedicated days for it rather than treating testing as leftover time at the end. What is cut to make the schedule fit is breadth of testing (large-scale load simulation), not testing of the concurrency-critical path itself.

6. Risk Assessment and Mitigation

Compressing the build into four weeks changes the risk profile from the original sixteen-week plan: schedule risk now sits alongside technical risk as a primary concern, since there is no slack sprint to absorb a slipped estimate.

Risk	Likelihood	Impact	Mitigation Strategy
Four-week window leaves no slack for schedule slip	High	High	MVP scope frozen before Sprint 1 (Section 3.1); daily build logs catch slippage within a day, not a sprint; mid-sprint check-in (Wednesday) triggers scope trimming against the deferred list rather than schedule overrun
Race condition allows double-booking of a seat	Medium	High	Distributed lock via Redisson before any hold is created; DB-level unique constraint on (show_id, seat_id, status=BOOKED) as a second line of defense; concurrency test suite simulating simultaneous requests for the same seat
Held seats never release due to a failed or missed expiry job	Medium	High	TTL set natively on the Redisson lock/hold key rather than relying solely on an application-level scheduler, so expiry does not depend on the app process staying alive
Redis/Redisson unavailability causes locking to fail open or fail closed incorrectly	Low	High	Documented failure mode from architecture review: on Redis unavailability, the system fails closed (rejects new holds) rather than allowing unsynchronized bookings; fallback pessimistic DB locking evaluated as a degraded-mode path
Insufficient time for broad-scale load or performance testing	Medium	Medium	Correctness-focused concurrency tests are mandatory in Sprints 2–3; large-scale JMeter load testing is explicitly deferred (Section 3.1) rather than attempted in a rushed, unreliable form
Scope creep from adding features beyond the SRD baseline	Medium	Medium	Sprint goals are checked against SRD functional requirements before being added to a sprint backlog; new ideas are logged as deferred/stretch items (Section 3.1), not merged into the active sprint
Payment gateway integration delays due to sandbox/API access issues	Low	Medium	Payment integration scheduled in Sprint 3, not on the critical path for the Sprint 1–2 seat-engine and concurrency demo, so a delay here does not block the earlier milestones
Reviewer questions whether design artifacts reflect independent understanding	Low	Medium	Daily build logs and sprint retrospectives (Section 4.2) create a running, dated record of hands-on decisions to support verbal defense

7. Integration and Testing Plan

Testing is layered to match the module boundaries defined in the Week 2 architecture document, and is treated as continuous throughout the Coding stage rather than a single phase at the end.

7.1 Testing Layers

Given the compressed schedule, testing is written alongside each sprint's code rather than saved for a dedicated phase; only regression and hardening are pulled into a distinct block, in Sprint 4. Load testing is deliberately kept lightweight and time-boxed rather than exhaustive, consistent with the scope decision in Section 3.1.

Layer	Scope	Tooling
Unit testing	Individual classes: seat grid logic, HashMap status lookups, sliding-window scan, PriorityQueue waitlist ordering, lazy heap deletion	JUnit 5, Mockito
Integration testing	Module interactions: seat service ↔ reservation service ↔ payment service, DB transactions, Redis/Redisson calls	Spring Boot Test, Testcontainers (PostgreSQL, Redis)
Concurrency testing	Simultaneous hold requests for the same seat and same show under simulated multi-user load — the one non-negotiable testing layer in this plan	Custom multi-threaded test harness; a small, time-boxed JMeter script for scripted concurrent HTTP load
System / end-to-end testing	Full user journey: browse → select seat → hold → pay → confirm, including expiry and waitlist promotion	Manual + scripted end-to-end test scenarios per SRD use cases
Regression testing	Re-run of prior sprints' test suites after each new module is merged, concentrated in Sprint 4	CI pipeline runs full suite on every merge to main

7.2 Integration Sequencing

Modules are integrated in the order in which they produce dependent data, minimizing the amount of mocking required at each step:

- Theatre / Screen / Show catalog service integrates first, since every other module depends on valid show and seat-layout data.
- Seat availability service integrates against the catalog service, validated with the 2D grid and HashMap lookup tests from Sprint 1.
- Reservation and locking service integrates against seat availability, validated with the concurrency test suite before payment integration begins, since a booking flow built on an unproven locking layer would need to be retested regardless.
- Payment service integrates last against the reservation service, since it only needs to consume a confirmed hold rather than participate in the concurrency-critical path.
- Notification (seat-unavailable / hold-expiring) hooks are integrated alongside the reservation service, since both are driven by the same state transitions.

7.3 Acceptance Criteria for Sign-Off

The implementation is considered ready for deployment (end of Sprint 4 / Week 6) when all of the following hold:

- All functional requirements listed in the Week 1 SRD have a corresponding passing automated test.
- The concurrency test suite shows zero double-bookings across repeated simulated concurrent-request runs against the same seat.
- Held-but-unpaid seats are automatically released within the 10-minute TTL window in all tested scenarios.
- The full CI pipeline (unit, integration, and regression suites) passes on the main branch.
- A deployment runbook exists and has been used to perform at least one clean deployment from a fresh environment.

8. Summary

This roadmap fits CineSeat's four one-week sprints following the Planning and Design work already completed in Weeks 1–2 — and prioritizes the highest-risk component, distributed seat-hold concurrency, early enough in that short window (Sprint 2) that problems can still be corrected before the final sprint. A fixed MVP scope, resource estimates sized to the true twenty-day budget, and risk mitigations that treat schedule slip as a first-class risk are intended to keep the plan honest about what four weeks can deliver, while still producing a system that is feature-complete against the Week 1 requirements and demonstrably correct under the concurrent-access conditions that motivated the project in the first place.